

Centro de formación:

Servicio Nacional De Aprendizaje

SENA

Programa de Formación:

Tecnólogo en Análisis y Desarrollo de Software

Proyecto Formativo:

TRUEQUE MATCH “Plataforma Colombiana de Intercambio Sin Dinero”

Evidencia GA7-220501096-AA4-EV01.

Taller sobre componentes front-end.

Aprendiz: Gerson Jonnathan López Oviedo

Ficha: 3186647

Instructor: Arnaldo Alfonso Montiel Brun

15 de junio, 20267

Contenido

1. INTRODUCCIÓN.....	3
2. OBJETIVOS.....	4
3. CARACTERÍSTICAS PRINCIPALES DE REACT.....	5
4. DESARROLLO DEL TALLER SOBRE COMPONENTES FRONT-END	5
5. ¿QUÉ SON CLASES EN REACT?	7
6. CARACTERÍSTICAS CLAVE DE LAS CLASES:.....	7
7. PRINCIPALES EVENTOS DE REACT	8
8. CLASIFICACIÓN DE LOS EVENTOS PRINCIPALES:	8
A. Eventos del Mouse (Mouse Events).....	8
B. Eventos de Formulario (Form Events)	9
C. Eventos del Teclado (Keyboard Events).....	9
9. MAPA CONCEPTUAL DE REACT	10
10. CONCLUSIONES	13

1. NTRODUCCIÓN

El desarrollo de aplicaciones web de alto rendimiento y con interfaces de usuario altamente interactivas ha evolucionado de forma acelerada, exigiendo que las metodologías tradicionales de manipulación del Modelo de Objetos del Documento (DOM) den paso a herramientas arquitectónicas más eficientes. En el ecosistema del desarrollo Front-End moderno, React se ha consolidado como la librería líder de JavaScript para la construcción de interfaces basadas en componentes reutilizables.

El propósito de este informe de investigación es profundizar en los conceptos fundamentales que componen esta tecnología. A lo largo del documento, se analizarán los mecanismos operativos de React, la naturaleza técnica de la sintaxis JSX, la transición de los componentes basados en clases hacia la adopción moderna de funciones y Hooks, y la gestión de eventos reactivos. Esta comprensión conceptual proveerá las bases técnicas necesarias para la codificación modular de interfaces dinámicas estables dentro del proyecto tecnológico formativo.

2. OBJETIVOS

Objetivo General

Desarrollar un taller conceptual e investigativo sobre los componentes e infraestructura del framework/librería React, estructurando sus características técnicas principales bajo las directrices académicas institucionales del SENA.

Objetivos Específicos

- Definir y contrastar de manera precisa la diferencia operativa entre la librería core de React y la extensión de sintaxis JSX.
- Explicar la naturaleza, ciclo de vida e implementación de las clases dentro de la arquitectura de componentes de React.
- Clasificar y ejemplificar el comportamiento de los principales eventos sintéticos del ecosistema React.
- Diseñar un mapa conceptual jerárquico que sintetice la arquitectura global de React para su asimilación técnica.

3. CARACTERÍSTICAS PRINCIPALES DE REACT

Antes de abordar el cuestionario técnico, es fundamental caracterizar los pilares informáticos que hacen de React una herramienta robusta para el Front-End:

1. **Arquitectura Basada en Componentes:** React divide la interfaz de usuario en bloques de código independientes, aislados y reutilizables. Cada componente controla su propio renderizado y lógica interna.
2. **Virtual DOM (DOM Virtual):** En lugar de actualizar directamente el DOM del navegador (lo cual consume excesivos recursos de cómputo), React genera una copia ligera en memoria. Cuando ocurren cambios, calcula la diferencia mínima requerida y actualiza únicamente los nodos del DOM real que fueron modificados, optimizando la velocidad de respuesta de la app.
3. **Flujo de Datos Unidireccional:** La información fluye exclusivamente de manera descendente, desde los componentes padres hacia los componentes hijos a través de propiedades (*props*). Esto incrementa la predictibilidad del software y facilita el rastreo de errores de estado.

4. DESARROLLO DEL TALLER SOBRE COMPONENTES FRONT-END

1. Diferencia entre React y JSX

Para comprender el desarrollo moderno en interfaces de usuario, es crítico discernir que **React** y **JSX** son dos tecnologías complementarias, pero con naturalezas informáticas totalmente distintas.

React

React es una librería de código abierto escrita en JavaScript, creada por Meta (Facebook). Su trabajo consiste puramente en gestionar la lógica de los componentes,

procesar el estado de la aplicación, calcular los cambios en el Virtual DOM y dictaminar cómo y cuándo debe reaccionar la interfaz de usuario ante las interacciones del operador.

JSX (JavaScript XML)

JSX no es un framework ni una librería; es una **extensión de la sintaxis estándar de JavaScript**. Permite a los desarrolladores escribir estructuras visuales idénticas al código HTML tradicional directamente dentro de los archivos de lógica JavaScript. El navegador web es incapaz de interpretar JSX de forma nativa; por ende, herramientas como Babel compilan este código JSX y lo transforman en llamadas de funciones puras de JavaScript (`React.createElement()`).

Ejemplo:

Imagine que usted está construyendo casas prefabricadas. **React** es el ingeniero de obra, el software de simulación y la grúa hidráulica que decide dónde encaja estructuralmente cada pared y cómo reaccionar si el terreno cambia. Por otro lado, **JSX** es el plano visual en 3D detallado que el cliente puede entender fácilmente. El plano (JSX) no construye la casa por sí mismo, sino que facilita al ingeniero (React) la lectura rápida de cómo debe lucir la estructura final antes de convertirla en concreto real (DOM nativo).

En código puro, la diferencia estructural se evidencia de la siguiente manera:

// Código escrito usando JSX (Sintaxis amigable para el desarrollador)

```
const elementoJSX = <h1 className="titulo">Hola, Trueque Match</h1>;
```

// Código compilado equivalente que procesa React internamente

```
const elementoReact = React.createElement(  
  'h1',  
  { className: 'titulo' },
```

'Hola, Trueque Match'

);

5. ¿QUÉ SON CLASES EN REACT?

En las versiones iniciales y maduras de React (hasta antes del año 2019), las **Clases** representaban el método imperativo para crear componentes complejos que requirieran almacenar un estado interno propio y conectarse de forma directa con el ciclo de vida del navegador.

Un componente de clase es esencialmente una clase de programación orientada a objetos (POO) de JavaScript que hereda o extiende las propiedades de la clase base `React.Component`. Obligatoriamente debe incluir un método llamado `render()`, el cual se encarga de retornar los elementos visuales (JSX) que serán dibujados en la pantalla.

6. CARACTERÍSTICAS CLAVE DE LAS CLASES:

- **Estado Local (`this.state`):** Permitían inicializar un objeto contenedor de datos mutables que obligaba al componente a redibujarse si dichos datos variaban.
- **Métodos de Ciclo de Vida:** Ofrecían funciones nativas automáticas como `componentDidMount()` (ejecutado cuando el componente aparece en pantalla, ideal para consultar bases de datos), `componentDidUpdate()` (ejecutado al alterarse una propiedad o estado) y `componentWillUnmount()` (ejecutado justo antes de destruir el componente).

Ejemplo:

Imagine una planta eléctrica tradicional con un tablero manual de palancas. La infraestructura de **Clases en React** funciona como esta planta. Para que opere adecuadamente, requiere configuraciones estrictas de herencia (conectar los cables a la base central), usar comandos específicos para leer sus medidores (`this.state.medidor`) y entrenar a un operario calificado que sepa qué hacer exactamente cuando la planta arranca, cuando se recalienta o cuando se apaga (métodos de ciclo de vida).

Nota técnica contemporánea: En el desarrollo moderno de software, las clases han sido relegadas a un segundo plano debido a la llegada de los **Componentes Funcionales con Hooks** (funciones simples de JavaScript equipadas con herramientas como `useState` y `useEffect`). Los componentes funcionales logran exactamente los mismos resultados que las clases, pero con un código mucho más limpio, corto y fácil de testear.

7. PRINCIPALES EVENTOS DE REACT

La gestión de interacciones del usuario (clics, pulsaciones de teclas, envíos de formularios) en React se realiza mediante un sistema denominado **Eventos Sintéticos (SyntheticEvents)**. React no conecta los listeners de eventos directamente al DOM real, sino que envuelve los eventos nativos del navegador en una capa de abstracción propia para asegurar que el código funcione de forma idéntica en cualquier navegador web del mercado (Chrome, Safari, Edge, Firefox).

A diferencia del HTML tradicional donde los eventos se escriben en minúsculas (v.g., `onclick`), en React se utiliza de forma obligatoria la nomenclatura **camelCase** (v.g., `onClick`) y se les pasa una función de JavaScript explícita en lugar de un string de texto.

8. CLASIFICACIÓN DE LOS EVENTOS PRINCIPALES:

A. Eventos del Mouse (Mouse Events)

Controlan las interacciones físicas del puntero o mouse del usuario sobre la interfaz.

- **onClick:** Se dispara cuando el usuario hace clic y suelta el botón sobre un elemento. Es el evento más utilizado para activar acciones.
- **onmouseenter / onmouseleave:** Se activan cuando el puntero del mouse entra o sale de los límites físicos de un componente (ideal para efectos visuales flotantes).

B. Eventos de Formulario (Form Events)

Monitorean el flujo de recolección de datos del usuario, críticos para procesos de autenticación o registros.

- **onChange:** Se ejecuta de manera interactiva en tiempo real cada vez que el valor de un campo de entrada de texto (`<input>`, `<textarea>`, `<select>`) es modificado por el usuario.
- **onSubmit:** Se dispara cuando el usuario envía un formulario (v.g., al presionar un botón de "Registrar"). Permite interceptar la acción para validar los datos antes de enviarlos al backend en PHP.

C. Eventos del Teclado (Keyboard Events)

Detectan las pulsaciones físicas sobre el hardware del teclado.

- **onKeyDown:** Se dispara en el instante en que el usuario presiona cualquier tecla.
- **onKeyUp:** Se ejecuta cuando el usuario libera la tecla presionada (útil para sistemas de búsqueda predictiva o atajos).

Ejemplo:

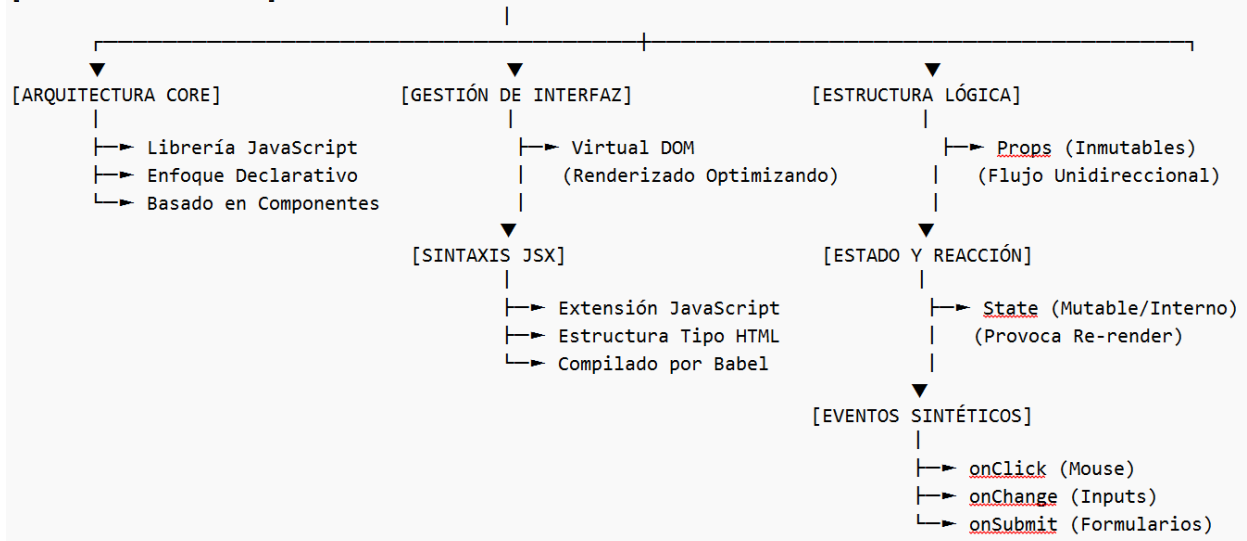
Imagine un auto de carreras moderno equipado con sensores de telemetría digital de alta sensibilidad. El pedal de aceleración, el botón de encendido y el volante representan los elementos físicos de la interfaz web. Los **Eventos en React** actúan como esos sensores digitales: en el instante milimétrico en que el piloto presiona el botón de encendido (`onClick`), el sensor encapsula el impulso eléctrico, estabiliza la señal para evitar interferencias climatológicas (abstracción sintética) y le ordena a la computadora del motor que inicie la marcha del vehículo de manera controlada.

9. MAPA CONCEPTUAL DE REACT

A continuación, se detalla la arquitectura relacional y el mapa conceptual de React estructurado mediante un diagrama de árbol de dependencias lógicas y jerárquicas:



[ECOCONCEPTO DE REACT]



10. CONCLUSIONES

- La implementación de React en la ingeniería de software Front-End simplifica radicalmente la creación de interfaces complejas mediante la modularización de componentes, permitiendo que la lógica visual sea altamente escalable y limpia gracias al uso extendido de la sintaxis JSX.
- Aunque los componentes basados en clases constituyeron los cimientos del ecosistema de React permitiendo la manipulación del estado y el ciclo de vida de los componentes, la industria tecnológica contemporánea ha migrado hacia arquitecturas funcionales con Hooks debido a su legibilidad de código superior y menor consumo de recursos del sistema.
- La optimización del rendimiento en aplicaciones interactivas en React está estrechamente ligada a su motor interno: la utilización estratégica del Virtual DOM combinada con la unificación estandarizada de eventos sintéticos asegura una experiencia de usuario fluida, multiplataforma y de alta velocidad de respuesta ante estímulos en tiempo real.